

RAG Fundamentals

03

Retrieval-Augmented Generation — give your AI grounded knowledge from real documents

6-Week Accelerator • Beginner to Intermediate • Python/JS Required

What You'll Learn



Understand what RAG is and why it's more powerful than a basic chatbot



Chunk documents correctly and create embeddings for vector search



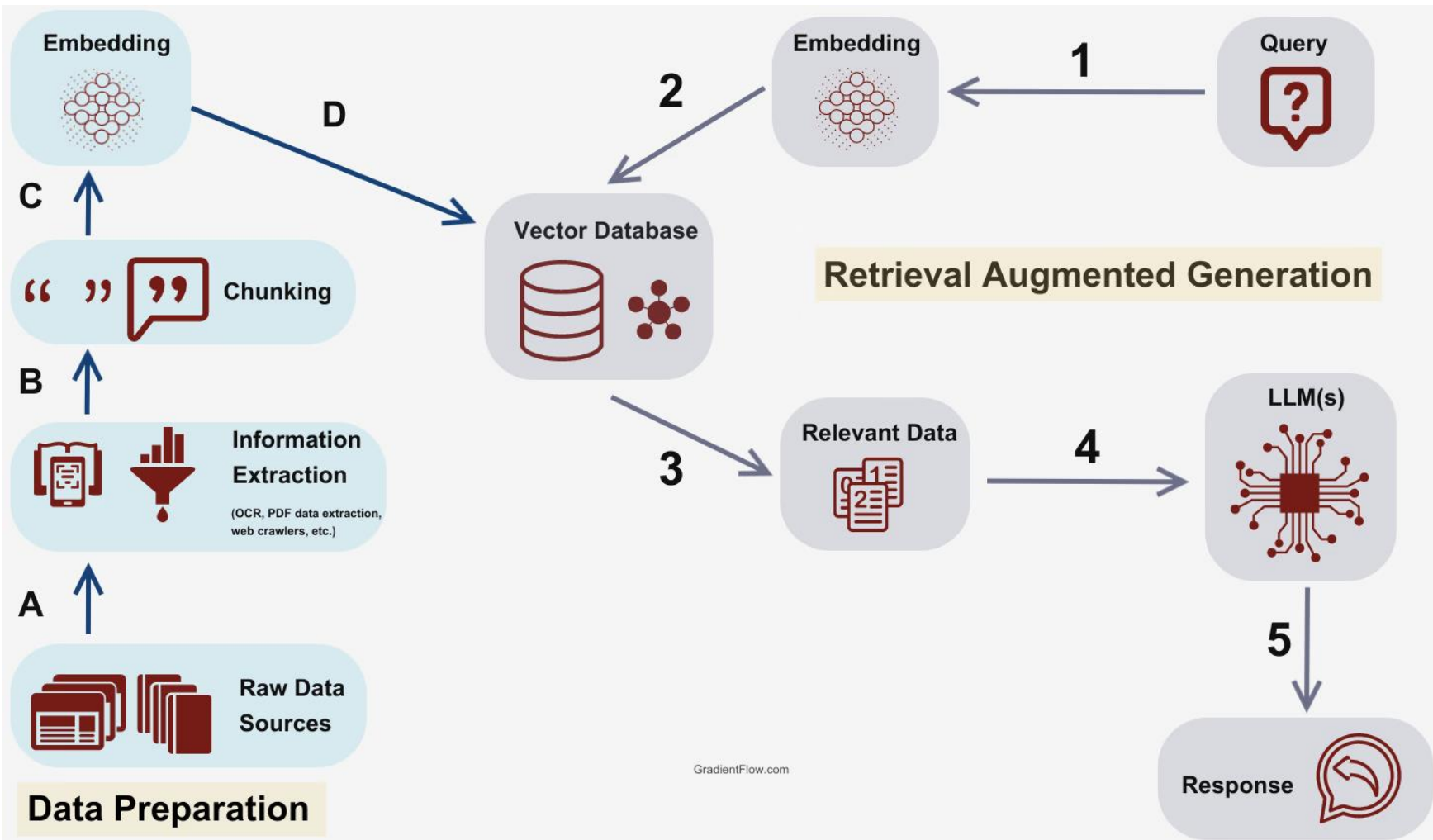
Set up a vector database and implement similarity retrieval



Inject retrieved context into LLM prompts to reduce hallucinations



Identify common RAG failure modes and how to avoid them



What Is RAG and Why Does It Matter?

Without RAG

Model only knows training data

Can hallucinate facts confidently

No access to your documents

Knowledge frozen at training cutoff

Can't cite sources reliably

With RAG

Retrieves relevant chunks at query time

Answers grounded in real documents

Works with your private data

Always uses current information

Can cite exact sources

The RAG Pipeline — Step by Step

01

Document Ingestion

Load PDFs, text files, web pages, or databases into your pipeline

02

Chunking

Split documents into 256-1024 token chunks with overlapping windows

03

Embedding

Convert each chunk to a vector using an embedding model (OpenAI, HuggingFace)

04

Vector Store

Index all vectors in a database: Chroma, Pinecone, Weaviate, or pgvector

05

Query Retrieval

Embed the user query, find top-k similar chunks via cosine similarity

06

Context Injection

Insert retrieved chunks into the LLM prompt as context before the question

Common RAG Failure Modes

✗ Chunking Too Large

Chunks over 1500 tokens dilute relevance.
Keep chunks focused and targeted.

✗ No Overlap

Without sliding window overlap, context
around chunk boundaries gets lost.

✗ Wrong Embedding Model

Mismatched domains cause poor
retrieval. Use domain-appropriate
embeddings.

✗ Hallucination Still Happens

Even with context, models may fabricate.
Always prompt to cite sources.

✗ Retrieval Irrelevance

Top-k results may not be semantically
relevant. Add reranking or filtering.

✗ Prompt Context Overflow

Too many chunks exceed the context
window. Limit retrieved chunks (3-5 max).

Document Q&A Assistant

Complete this practical output before moving to the next module. Add it to your GitHub portfolio.

WHAT SUCCESS LOOKS LIKE

✓ Completed and functional — not just a screenshot or stub

✓ Committed to a public GitHub repository with clear README

✓ Reflects concepts from this module — shows applied understanding

✓ Peer-reviewable — someone else could understand it in 5 minutes

Module Summary

Key Takeaways

- RAG = retrieve → inject context → generate grounded response
- Chunking strategy heavily impacts retrieval quality
- Embeddings map meaning to vectors — similarity search is key
- Always prompt models to cite sources in RAG apps
- Monitor retrieval quality as closely as generation quality

Up Next

Module 4

AI Security for Builders

Complete this module's practical output before moving forward.